

Name: Manaswi Kabadi

Class: D20C

Roll No.: 15

Blockchain Experiment 5

Aim:

Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development.

Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values.

Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- Function Visibility defines who can access a function:
 - **public:** available both inside and outside the contract.
 - **private:** only accessible within the same contract.
 - **internal:** accessible within the contract and its child contracts.

- external: can be called only by external accounts or other contracts.
- Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- if-else allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- Loops (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- Arrays: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- Mappings: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- Structs: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- Enums: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- storage: Data stored permanently on the blockchain. Examples: state variables.
- memory: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- calldata: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory.

Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

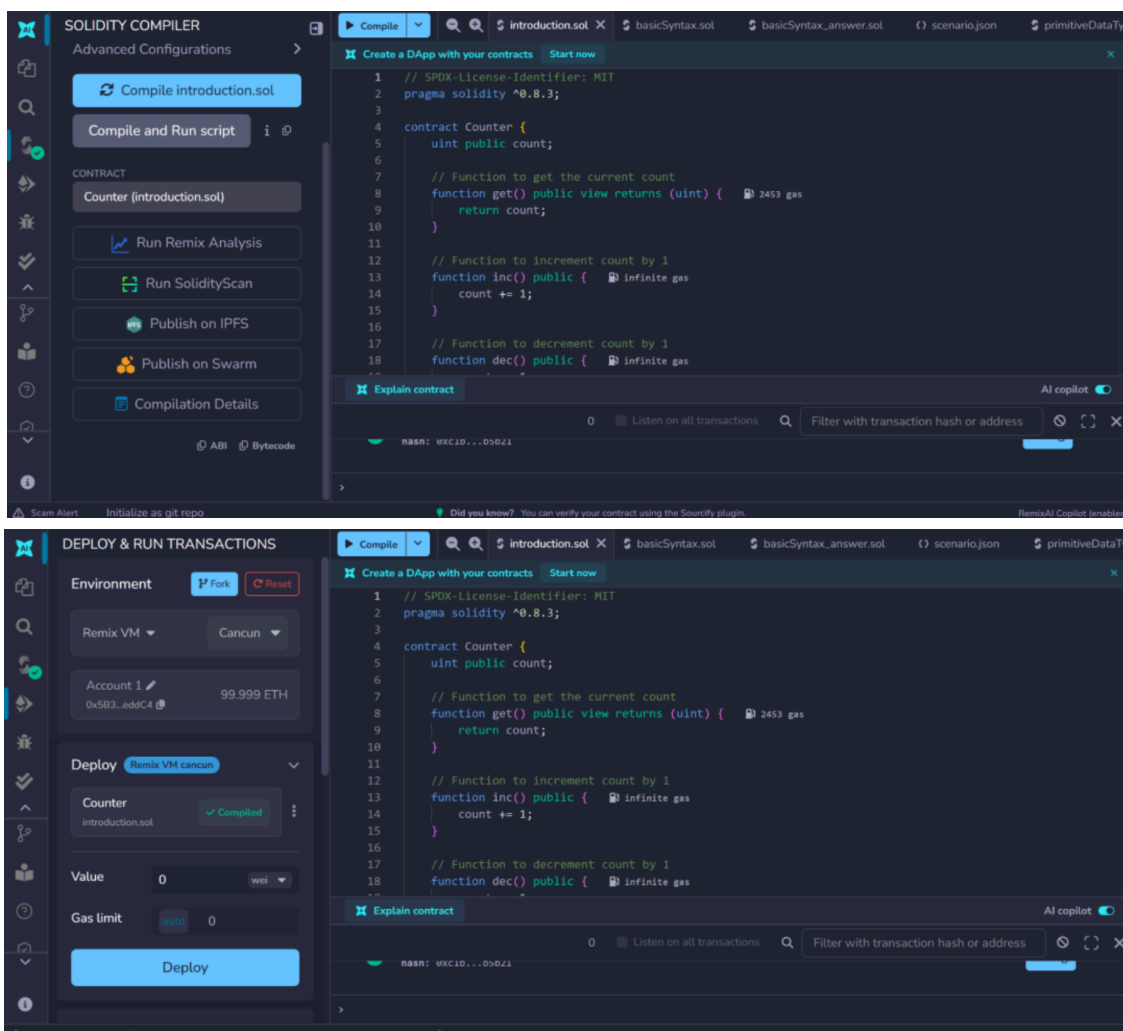
- Ether and Wei: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10¹⁸ Wei). This ensures high precision in financial

transactions.

- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Output:

Tutorial 1:



DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

99.999 ETH

Select a function to interact with...

Low level interaction

+

• get

uint256: 0

Call

Balance

ETH

Compile

introduction.sol

basicSyntax.sol

basicSyntax_answer.sol

scenario.json

primitiveDataTy

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19    }
20 }
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

Did you know? You can verify your contract using the Sourcify plugin.

RemixAI Copilot (enabled)

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

99.999 ETH

Select a function to interact with...

Low level interaction

+

• inc

Value

1

wei

Gas limit

auto

0

Transact

Compile

introduction.sol

basicSyntax.sol

basicSyntax_answer.sol

scenario.json

primitiveDataTy

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19    }
20 }
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

Did you know? You can verify your contract using the Sourcify plugin.

RemixAI Copilot (enabled)

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

99.999 ETH

Functions

Select a function to interact with...

Low level interaction

+

• count

uint256: 1

Call

Compile

introduction.sol

basicSyntax.sol

basicSyntax_answer.sol

scenario.json

primitiveDataTy

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19    }
20 }
```

Explain contract

AI copilot

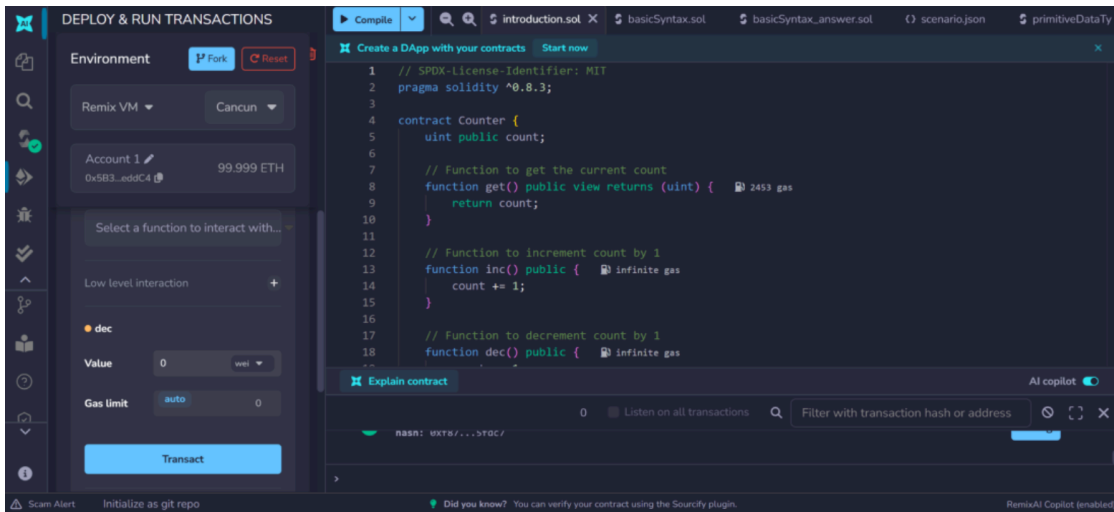
0

Listen on all transactions

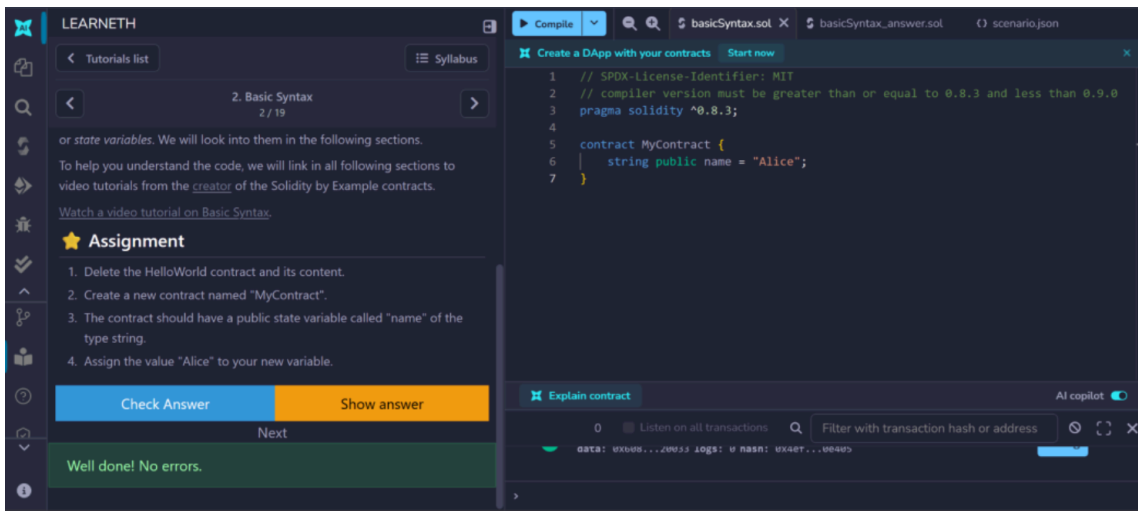
Filter with transaction hash or address

Did you know? You can verify your contract using the Sourcify plugin.

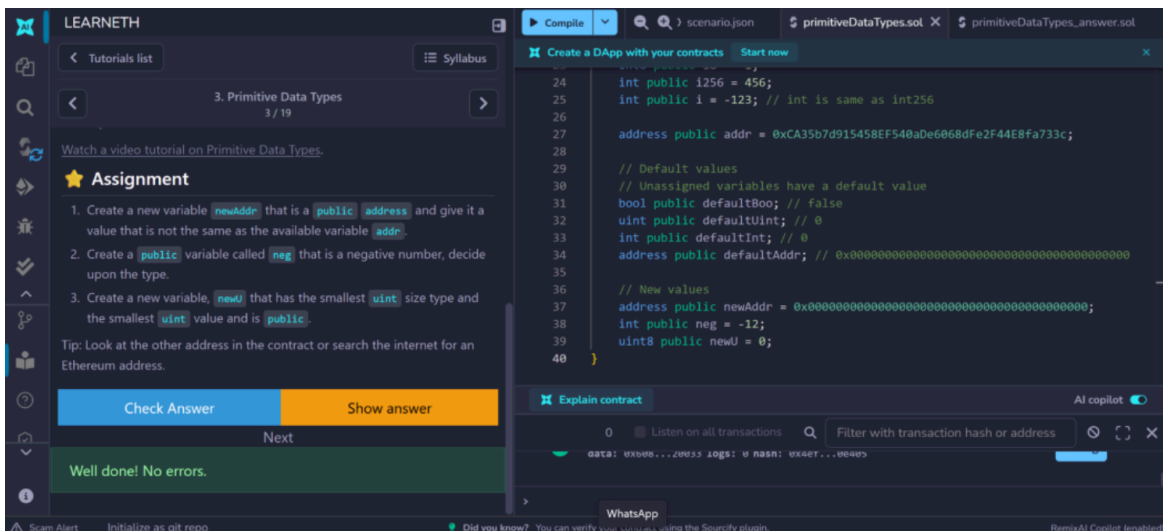
RemixAI Copilot (enabled)



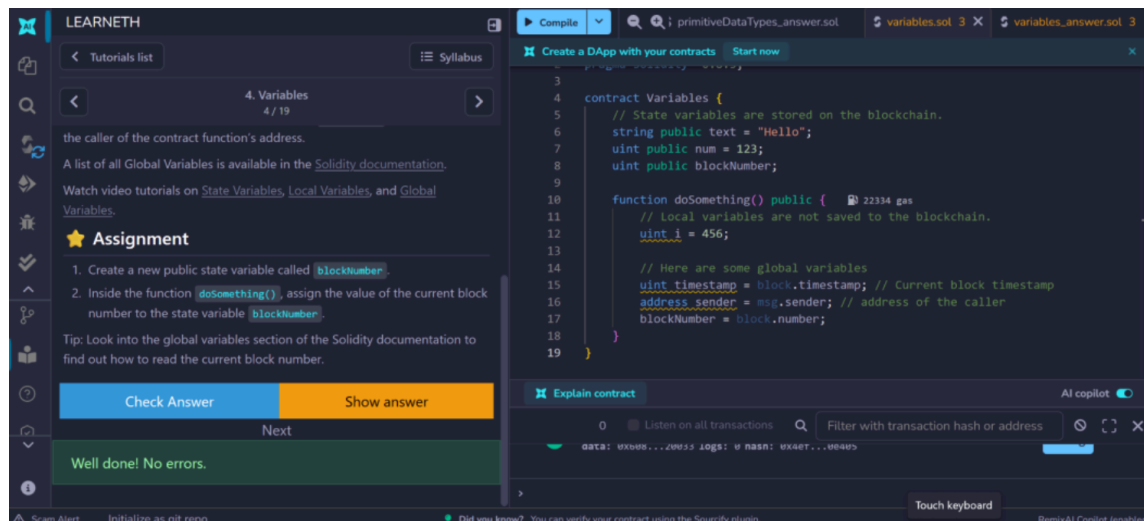
Tutorial 2:



Tutorial 3:



Tutorial 4:



LEARNETH

Tutorials list

4. Variables
4 / 19

the caller of the contract function's address.
A list of all Global Variables is available in the [Solidity documentation](#).
Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ **Assignment**

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public {
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

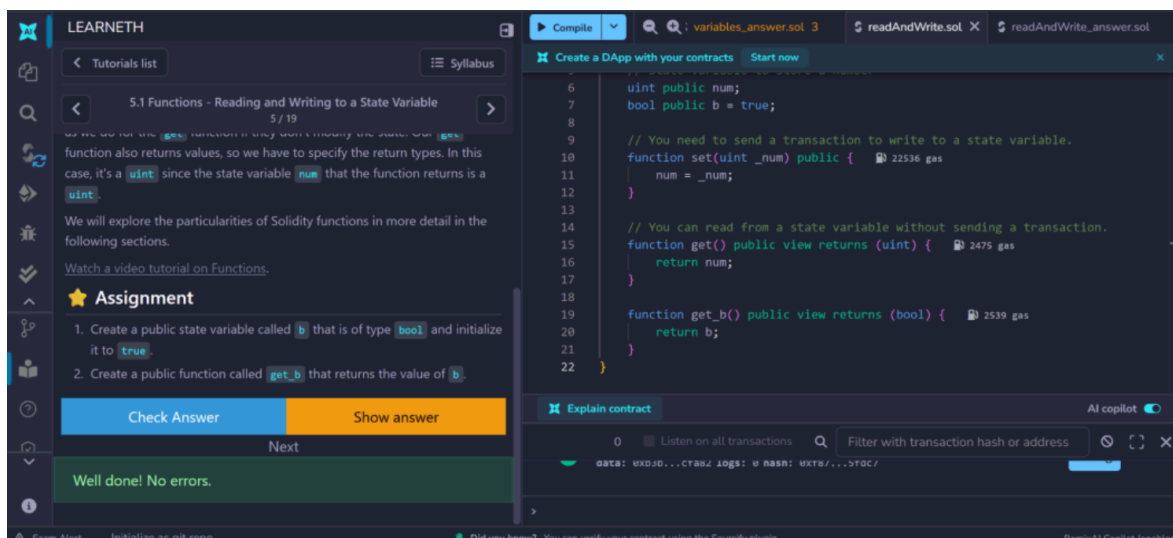
Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...0000 logs: 0 hash: 0x000...0000

Touch keyboard

Tutorial 5:



LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable
5 / 19

We will explore the particularities of Solidity functions in more detail in the following sections.
Watch a video tutorial on [Functions](#).

★ **Assignment**

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer Show answer

Next

Well done! No errors.

Create a DApp with your contracts Start now

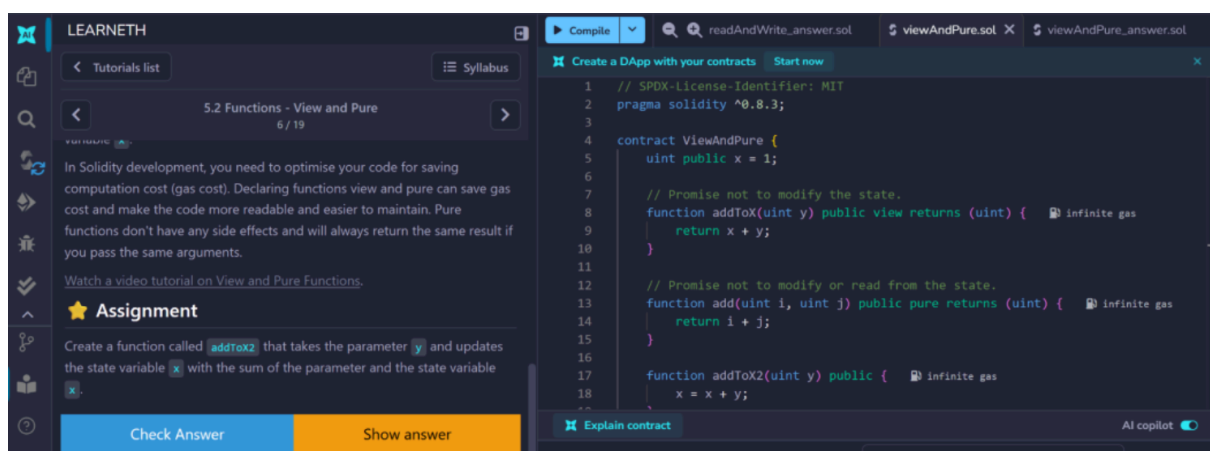
```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 uint public num;
5 bool public b = true;
6
7 // You need to send a transaction to write to a state variable.
8 function set(uint _num) public {
9     num = _num;
10 }
11
12 // You can read from a state variable without sending a transaction.
13 function get() public view returns (uint) {
14     return num;
15 }
16
17 function get_b() public view returns (bool) {
18     return b;
19 }
20
21
22 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...0000 logs: 0 hash: 0x000...0000

Tutorial 6:



LEARNETH

Tutorials list

5.2 Functions - View and Pure
6 / 19

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.
Watch a video tutorial on [View and Pure Functions](#).

★ **Assignment**

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

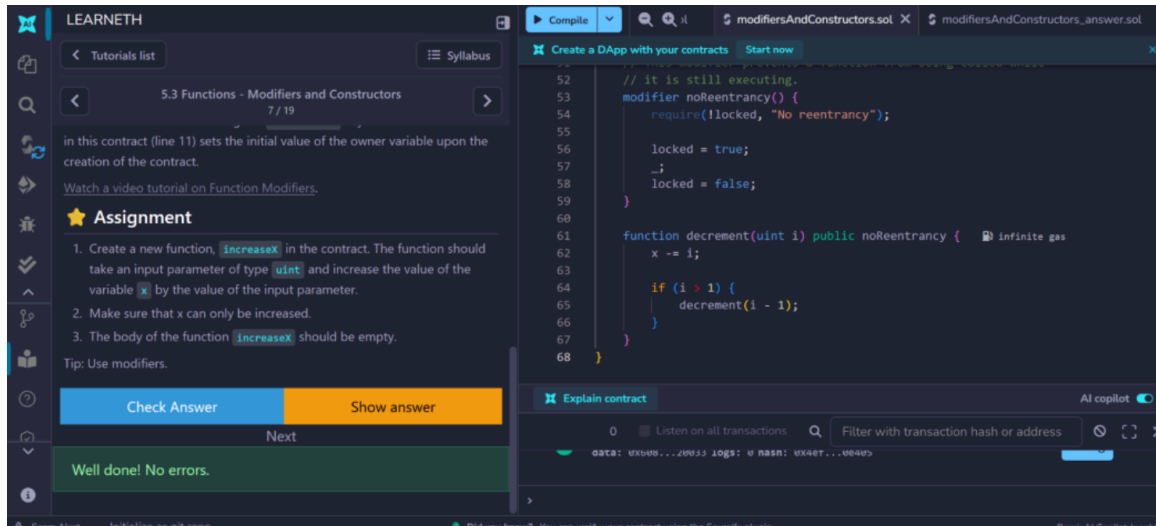
Check Answer Show answer

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addToX2(uint y) public {
18        x = x + y;
19    }
20 }
```

Explain contract AI copilot

Tutorial 7:



LEARNETH

Tutorials list | Syllabus

5.3 Functions - Modifiers and Constructors (7 / 19)

in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

★ **Assignment**

1. Create a new function, `increaseX`, in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts [Start now](#)

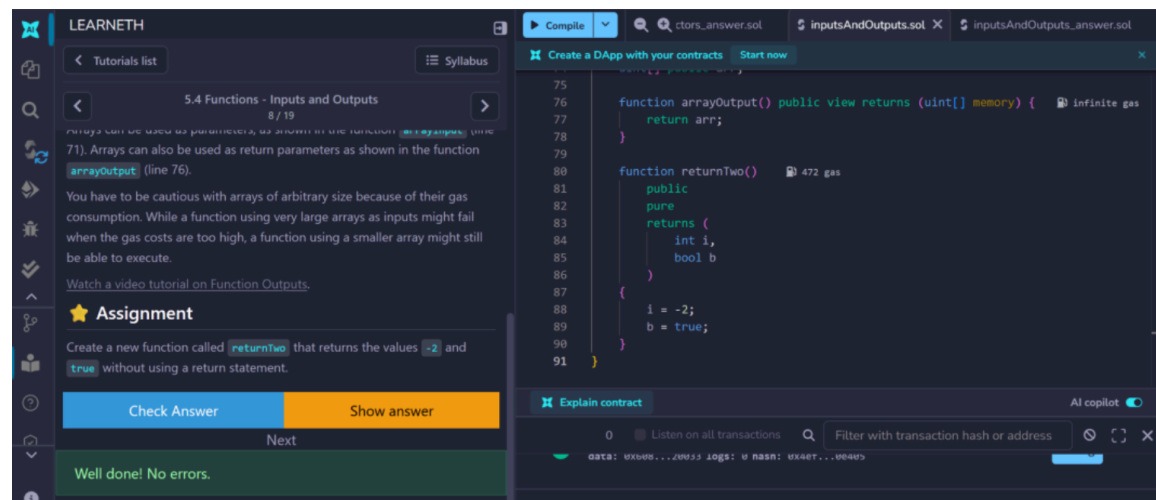
```
52 // it is still executing.
53 modifier noReentrancy() {
54     require(!locked, "No reentrancy");
55
56     locked = true;
57     _;
58     locked = false;
59 }
60
61 function decrement(uint i) public noReentrancy { Infinite gas
62     x -= i;
63
64     if (i > 1) {
65         decrement(i - 1);
66     }
67 }
68 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...00000 logs: 0 hash: 0x40e...00000

Tutorial 8:



LEARNETH

Tutorials list | Syllabus

5.4 Functions - Inputs and Outputs (8 / 19)

Arrays can be used as parameters, as shown in the function `arrayOutput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

★ **Assignment**

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts [Start now](#)

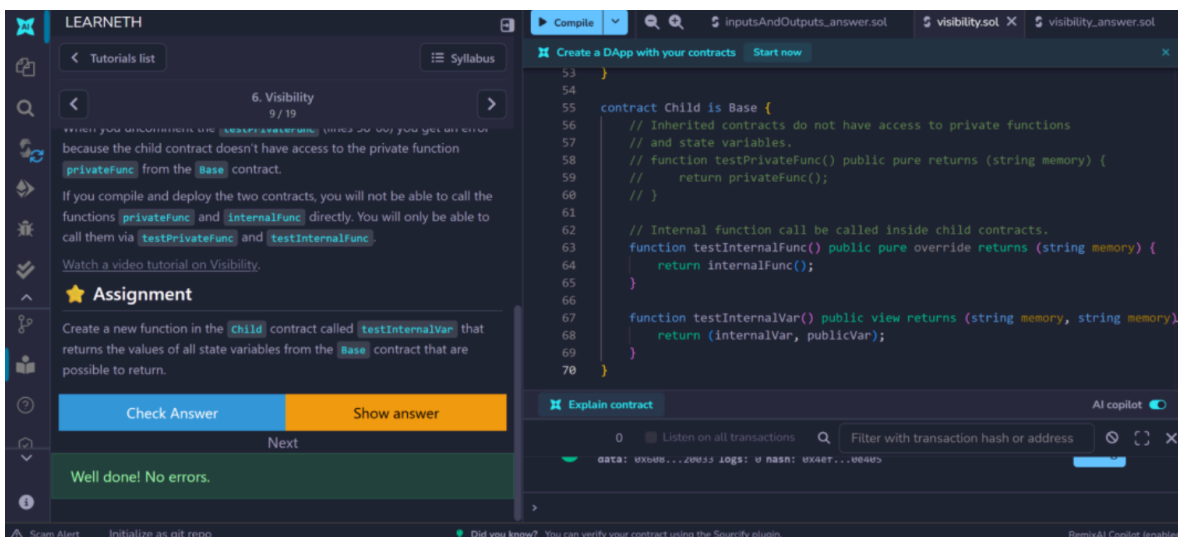
```
75
76 function arrayOutput() public view returns (uint[] memory) { Infinite gas
77     return arr;
78 }
79
80 function returnTwo() 472 gas
81     public
82     pure
83     returns (
84         int i,
85         bool b
86     )
87 {
88     i = -2;
89     b = true;
90 }
91 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...00000 logs: 0 hash: 0x40e...00000

Tutorial 9:



LEARNETH

Tutorials list | Syllabus

6. Visibility (9 / 19)

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

★ **Assignment**

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts [Start now](#)

```
53
54
55 contract Child is Base {
56     // Inherited contracts do not have access to private functions
57     // and state variables.
58     // function testPrivateFunc() public pure returns (string memory) {
59     //     return privateFunc();
60     // }
61
62     // Internal function call be called inside child contracts.
63     function testInternalFunc() public pure override returns (string memory) {
64         return internalFunc();
65     }
66
67     function testInternalVar() public view returns (string memory, string memory) {
68         return (internalVar, publicVar);
69     }
70 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...00000 logs: 0 hash: 0x40e...00000

Tutorial 10:

LEARNETH

Tutorials list

7.1 Control Flow - If/Else
10 / 19

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.
[Watch a video tutorial on the If/Else statement.](#)

★ **Assignment**

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts [Start now](#)

```
12 }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // return 2;
20
21     // shorthand way to write if / else statement
22     return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck(uint y) public pure returns (bool) {
26     return y % 2 == 0 ? true : false;
27 }
28 }
```

Explain contract

0 ☐ Listen on all transactions

DATA: 0x000...20000 LOGS: 0x000...00000

Tutorial 11:

LEARNETH

Tutorials list

7.2 Control Flow - Loops
11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

[Watch a video tutorial on Loop statements.](#)

★ **Assignment**

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts [Start now](#)

```
11     continue;
12 }
13 if (i == 5) {
14     // Exit loop with break
15     break;
16 }
17 count++;
18 }
19
20 // while loop
21 uint j;
22 while (j < 10) {
23     j++;
24 }
25 }
26
27 }
```

Explain contract

0 ☐ Listen on all transactions

DATA: 0x000...20000 LOGS: 0x000...00000

Tutorial 12:

LEARNETH

Tutorials list

8.1 Data Structures - Arrays
12 / 19

the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the `length` member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ **Assignment**

- Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
- Change the `getArr()` function to return the value of `arr3`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts [Start now](#)

```
50 // Remove the last element
51 arr.pop();
52 }
53
54 function test() public {
55     arr.push(1);
56     arr.push(2);
57     arr.push(3);
58     arr.push(4);
59     // [1, 2, 3, 4]
60
61     remove(1);
62     // [1, 4, 3]
63
64     remove(2);
65     // [1, 4]
66 }
67
68 }
```

Explain contract

0 ☐ Listen on all transactions

DATA: 0x000...20000 LOGS: 0x000...00000

Tutorial 13:

LEARNETH

Tutorials list | Syllabus

8.2 Data Structures - Mappings 13 / 19

which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Compile | `ys.sol` | `arrays_answer.sol` | `mappings.sol` | `mappings_answer.sol`

Create a DApp with your contracts Start now

```
30 // You can get values from a nested mapping
31 // even when it is not initialized
32 return nested[_addr1][_i];
33
34
35 function set(
36     address _addr1,
37     uint _i,
38     bool _boo
39 ) public {
40     nested[_addr1][_i] = _boo;
41 }
42
43 function remove(address _addr1, uint _i) public {
44     delete nested[_addr1][_i];
45 }
46 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...0000 logs: 0 hash: 0x40e...0040e

Tutorial 14:

LEARNETH

Tutorials list | Syllabus

8.3 Data Structures - Structs 14 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ **Assignment**

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Compile | `pings.sol` | `mappings_answer.sol` | `structs.sol` | `structs_answer.sol`

Create a DApp with your contracts Start now

```
35
36 // update text
37 function update(uint _index, string memory _text) public {
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public {
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47
48 function remove(uint _index) public {
49     delete todos[_index];
50 }
51 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...0000 logs: 0 hash: 0x40e...0040e

Tutorial 15:

LEARNETH

Tutorials list | Syllabus

8.4 Data Structures - Enums 15 / 19

providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ **Assignment**

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Compile | `structs.sol` | `structs_answer.sol` | `enums.sol` | `enums_answer.sol`

Create a DApp with your contracts Start now

```
31
32
33 // Update status by passing uint into input
34 function set(Status _status) public {
35     status = _status;
36 }
37
38 // You can update to a specific enum like this
39 function cancel() public {
40     status = Status.Canceled;
41 }
42
43 // delete resets the enum to its first value, 0
44 function reset() public {
45     delete status;
46 }
47 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x000...0000 logs: 0 hash: 0x40e...0040e

Tutorial 16:

LEARNETH

Tutorials list

9. Data Locations
16 / 19

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

```
16 myStruct storage myStruct = myStructs[1];
17 myStruct.foo = 4;
18 // create a struct in memory
19 MyStruct memory myMemStruct = MyStruct(0);
20 MyStruct memory myMemStruct2 = myMemStruct;
21 myMemStruct2.foo = 1;
22
23 MyStruct memory myMemStruct3 = myStruct;
24 myMemStruct3.foo = 3;
25 return (myStruct, myMemStruct2, myMemStruct3);
26
27
28 function _f() {
29     // undefined gas
30     uint[] storage _arr,
31     mapping(uint => address) storage _map,
32     MyStruct storage _myStruct
33     internal {
34         // do something with storage variables
35     }
36 }
```

Tutorial 17:

LEARNETH

Tutorials list

10.1 Transactions - Ether and Wei
17 / 19

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneWei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

Tutorial 18:

LEARNETH

Tutorials list

10.2 Transactions - Gas and Gas Price
18 / 19

refunded.

Learn more about `gas` on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas Profiler` to check for the gas cost of transactions.

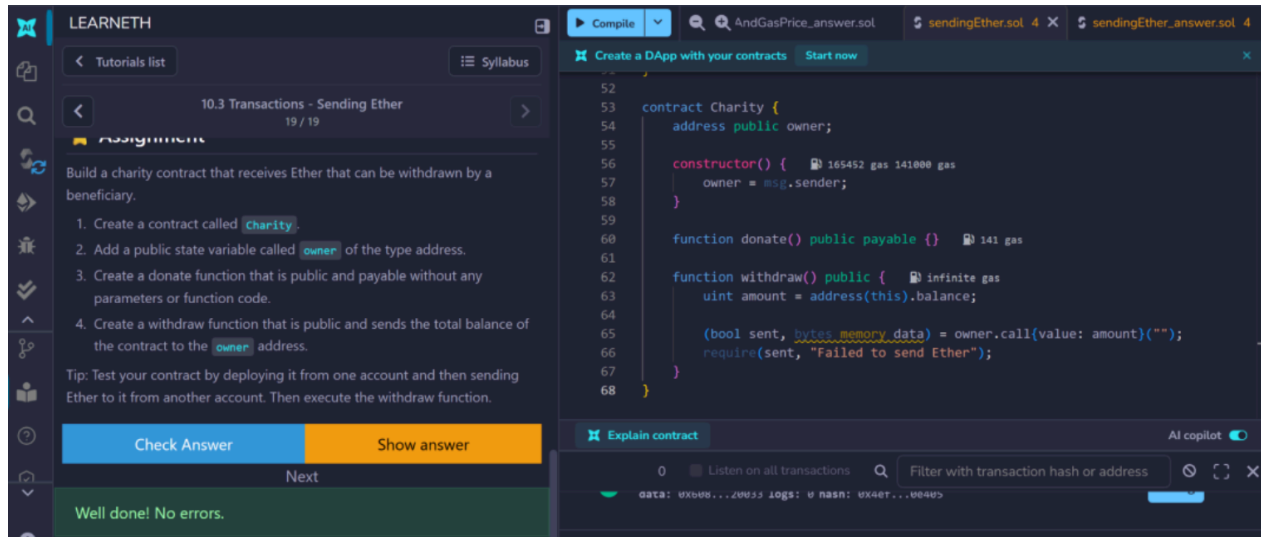
Check Answer Show answer

Next

Well done! No errors.

```
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // infinite gas
13        // Here we run a loop until all of the gas are spent
14        // and the transaction fails
15        while (true) {
16            i += 1;
17        }
18    }
19 }
```

Tutorial 19:



Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.